



Universidad Tecnológica de Bolívar

Universidad Tecnológica de Bolívar

Estimación del periodo de oscilación de un péndulo simple

Actividad 2 — Segundo corte

Sensado y Modelado de Sistemas Físicos

Estudiantes:

German De Armas Castaño
Angel Vega Rodriguez
Michael Taboada Naranjo

Profesor:

David Sierra Porta

23 de abril de 2026

Índice

1. Introducción	2
1.1. Planteamiento del problema	2
1.2. Objetivos	2
1.3. Parámetros de la simulación	3
2. Fundamentos Teóricos	3
2.1. Ecuación de movimiento del péndulo simple	3
2.2. Corrección para ángulos grandes (Borda)	3
2.3. Péndulo amortiguado	4
2.4. Reconstrucción angular desde coordenadas de píxeles	4
2.5. Transformada Rápida de Fourier (FFT)	4
2.6. Reconstrucción por series de Fourier	5
3. Implementación en el Notebook	5
3.1. Módulo 1: Detección de la masa (<code>detect_pendulum_mass</code>)	5
3.2. Módulo 2: Tracking y extracción de series temporales	6
3.3. Módulo 3: Reconstrucción de la trayectoria angular	6
3.4. Módulo 4: Estimación del periodo — Tres métodos	7
3.4.1. Método 1: Cruces por cero	7
3.4.2. Método 2: Detección de picos	7
3.4.3. Método 3: Transformada de Fourier (FFT)	8
3.5. Módulo 5: Reconstrucción por series de Fourier	9
3.6. Módulo 6: Comparación con valores teóricos	10
3.7. Módulo 7: Análisis de amortiguamiento	11
4. Aplicación Web en el Bento Launcher	12
4.1. Arquitectura	12
4.2. Lanzamiento desde el Bento Launcher	13
4.3. Carga del video y vista previa	14
4.4. Calibración HSV interactiva	14
4.5. Procesamiento y tracking en tiempo real	15
4.6. Resultados consolidados	15
5. Discusión	16
5.1. Concordancia entre métodos	16
5.2. Efecto del ángulo inicial grande	16
5.3. Análisis espectral	16
5.4. Amortiguamiento	17
5.5. Fuentes de error	17
6. Conclusiones	17

1. Introducción

El presente reporte documenta el diseño, implementación y validación de un sistema automatizado para la estimación del **periodo de oscilación** de un péndulo simple a partir del procesamiento de video. El proyecto se desarrolló en dos fases:

1. **Fase de exploración y prototipado** — implementada en el notebook: `pendulum_analyzer.ipynb`. Aquí se diseñaron y validaron los algoritmos de visión computacional, reconstrucción angular, detección de picos, análisis espectral (FFT) y ajuste de amortiguamiento.
2. **Fase de productización** — los hallazgos del notebook se encapsularon en una aplicación web interactiva (React + FastAPI) integrada al ecosistema **Bento Launcher**.

1.1. Planteamiento del problema

El péndulo simple es uno de los sistemas dinámicos más estudiados en física. Su periodo de oscilación depende de la longitud de la cuerda y de la aceleración gravitatoria, lo que lo convierte en un instrumento clásico para estimar g . Sin embargo, medir el periodo manualmente presenta limitaciones:

- La **latencia de reacción humana** al accionar un cronómetro introduce errores del orden de ± 200 ms.
- Para ángulos grandes ($\theta_0 > 15^\circ$), la aproximación lineal $\sin \theta \approx \theta$ deja de ser válida, y el periodo real difiere del teórico simple.
- El **amortiguamiento** por fricción causa que la amplitud decaiga con el tiempo, dificultando la medición de periodos consistentes.

Un sistema de visión computacional combinado con análisis espectral permite superar estas limitaciones, extrayendo la dinámica completa del sistema a partir de un video.

1.2. Objetivos

1. Implementar un algoritmo de segmentación por color (HSV) para rastrear la masa del péndulo frame a frame.
2. Reconstruir la trayectoria angular $\theta(t)$ a partir de las coordenadas del centroide y la posición conocida del pivote.
3. Estimar el periodo de oscilación mediante tres métodos independientes: cruces por cero, detección de picos y Transformada Rápida de Fourier (FFT).
4. Comparar los resultados experimentales con los valores teóricos (aproximación lineal y corrección de Borda).

5. Analizar el amortiguamiento del sistema mediante ajuste exponencial de la envolvente de amplitudes.
6. Encapsular el pipeline en una aplicación web desplegable desde el Bento Launcher.

1.3. Parámetros de la simulación

El video analizado fue generado mediante una simulación numérica con los siguientes parámetros conocidos:

Parámetro	Símbolo	Valor
Aceleración gravitatoria	g	9.81 m/s ²
Longitud de la cuerda	L	1.0 m
Ángulo inicial	θ_0	45°
Coefficiente de fricción	μ	0.05
Frames por segundo	FPS	30
Escala espacial	—	400 px/m

Cuadro 1: Parámetros conocidos de la simulación del péndulo.

2. Fundamentos Teóricos

2.1. Ecuación de movimiento del péndulo simple

La ecuación diferencial que gobierna el movimiento de un péndulo simple ideal (sin fricción) es:

$$\ddot{\theta} + \frac{g}{L} \sin \theta = 0 \quad (1)$$

Para oscilaciones de pequeña amplitud ($\theta \ll 1$ rad), se aplica la aproximación $\sin \theta \approx \theta$, obteniendo un oscilador armónico simple con periodo:

$$T_{\text{lineal}} = 2\pi \sqrt{\frac{L}{g}} \quad (2)$$

2.2. Corrección para ángulos grandes (Borda)

Para ángulos iniciales grandes, el periodo real es mayor que el predicho por la ecuación (2). La corrección de Borda expande el periodo exacto en serie de potencias del ángulo inicial:

$$T_{\text{Borda}} \approx T_{\text{lineal}} \left(1 + \frac{1}{16} \theta_0^2 + \frac{11}{3072} \theta_0^4 + \dots \right) \quad (3)$$

Para $\theta_0 = 45^\circ \approx 0,785$ rad, la corrección es del orden del 4-5 %, lo que la hace significativa y necesaria para una comparación rigurosa.

2.3. Péndulo amortiguado

Al incluir fricción proporcional a la velocidad angular, la ecuación de movimiento se modifica:

$$\ddot{\theta} + 2\gamma\dot{\theta} + \frac{g}{L} \sin \theta = 0 \quad (4)$$

donde γ es el coeficiente de amortiguamiento. La amplitud de las oscilaciones decae exponencialmente:

$$A(t) = A_0 e^{-\gamma t} \quad (5)$$

La constante de tiempo $\tau = 1/\gamma$ indica el tiempo en que la amplitud se reduce a $1/e$ de su valor inicial.

2.4. Reconstrucción angular desde coordenadas de píxeles

Conocida la posición del pivote (x_p, y_p) y la posición de la masa (x_m, y_m) en cada frame, el ángulo respecto a la vertical se calcula como:

$$\theta = \arctan\left(\frac{x_m - x_p}{y_m - y_p}\right) \quad (6)$$

donde se utiliza `arctan2` para preservar el signo y el cuadrante correcto.

2.5. Transformada Rápida de Fourier (FFT)

La FFT descompone una señal discreta $x[n]$ de N muestras en sus componentes de frecuencia:

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-i2\pi kn/N}, \quad k = 0, 1, \dots, N-1 \quad (7)$$

El **espectro de potencia** $|X[k]|^2$ revela las frecuencias dominantes de la señal. La frecuencia fundamental f_0 corresponde al pico de mayor magnitud, y el periodo se obtiene como $T = 1/f_0$.

Para reducir el **leakage espectral** (dispersión de energía causada por la truncación de la señal), se aplica una **ventana de Hanning** antes de la FFT:

$$w[n] = \frac{1}{2} \left(1 - \cos\left(\frac{2\pi n}{N-1}\right) \right) \quad (8)$$

2.6. Reconstrucción por series de Fourier

La señal original puede reconstruirse utilizando los K armónicos más significativos:

$$\theta_{\text{rec}}(t) = \frac{a_0}{2} + \sum_{k=1}^K [a_k \cos(2\pi f_k t) + b_k \sin(2\pi f_k t)] \quad (9)$$

Esto permite evaluar cuántos términos son necesarios para capturar la dinámica del sistema y cuantificar el error de reconstrucción mediante el RMSE.

3. Implementación en el Notebook

El notebook `pendulum_analyzer.ipynb` estructura el análisis en ocho módulos secuenciales. A continuación se detalla cada uno.

3.1. Módulo 1: Detección de la masa (`detect_pendulum mass`)

La función de detección sigue el mismo pipeline validado en el proyecto del coeficiente de restitución:

1. Conversión BGR $\rightarrow$ HSV.
2. Umbralizado por rango: $H \in [0, 15]$, $S \in [80, 255]$, $V \in [80, 255]$ (masa roja).
3. Limpieza morfológica (apertura + cierre, kernel elíptico 5×5).
4. Búsqueda de contornos y selección del mayor.
5. Cálculo del centroide (c_x, c_y) mediante momentos espaciales.

```

1 def detect_pendulum_mass(frame, lower_hsv, upper_hsv, min_area
  =100):
2     hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
3     mask = cv2.inRange(hsv, lower_hsv, upper_hsv)
4
5     kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
6     mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel)
7     mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, kernel)
8
9     contours, _ = cv2.findContours(

```

```

10     mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
11     if not contours:
12         return None
13
14     largest = max(contours, key=cv2.contourArea)
15     if cv2.contourArea(largest) < min_area:
16         return None
17
18     M = cv2.moments(largest)
19     if M["m00"] == 0:
20         return None
21
22     cx = int(M["m10"] / M["m00"])
23     cy = int(M["m01"] / M["m00"])
24     return (cx, cy)

```

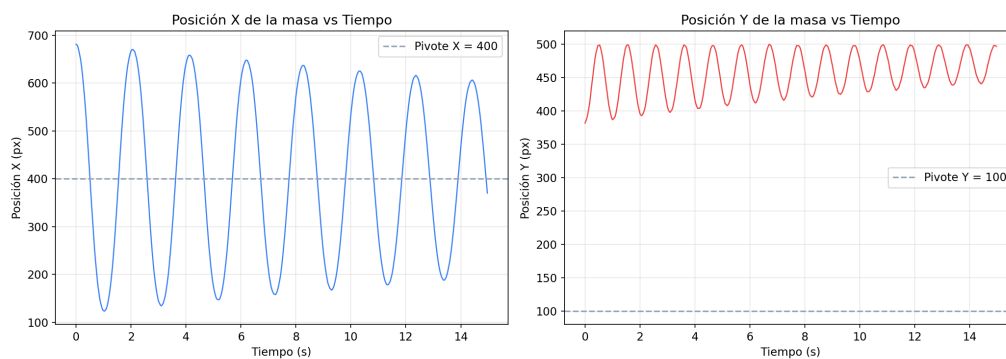
Listing 1: Función de detección de la masa del péndulo.

3.2. Módulo 2: Tracking y extracción de series temporales

El video se recorre frame a frame. Para cada detección exitosa se registra el instante temporal $t = \text{frame_idx}/\text{FPS}$ y las coordenadas (x, y) del centroide. El resultado son tres arrays NumPy: `times`, `x_positions` y `y_positions`.

3.3. Módulo 3: Reconstrucción de la trayectoria angular

Con las coordenadas del pivote conocidas (centro superior del lienzo, (400, 100) px), se aplica la ecuación (6) para obtener $\theta(t)$ en radianes.

Figura 1: Posición $X(t)$ e $Y(t)$ de la masa del péndulo extraídas del video.

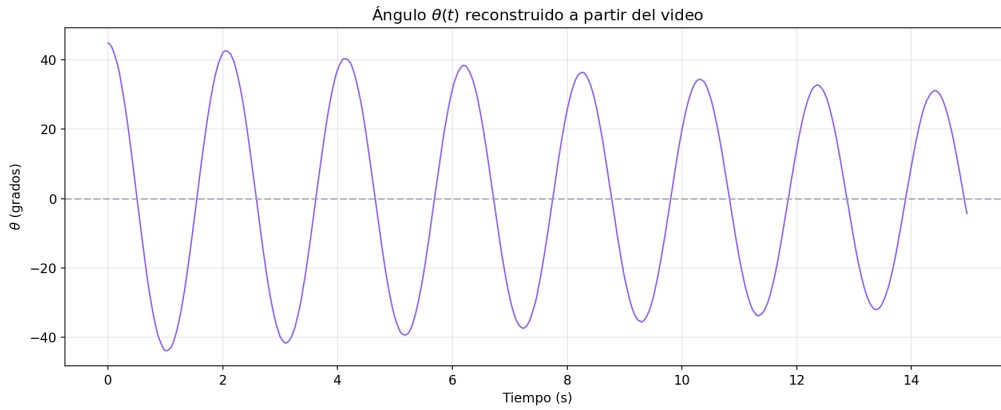


Figura 2: Ángulo $\theta(t)$ reconstruido a partir de las coordenadas del centroide y el pivote.

3.4. Módulo 4: Estimación del periodo — Tres métodos

Se implementaron tres métodos independientes para estimar el periodo de oscilación, permitiendo una validación cruzada de los resultados.

3.4.1. Método 1: Cruces por cero

Se detectan los instantes en que $\theta(t)$ cambia de signo. Cada cruce corresponde a un semi-periodo, por lo que:

$$T_{ZC} = 2 \cdot \overline{\Delta t_{\text{cruces}}} \quad (10)$$

3.4.2. Método 2: Detección de picos

Se aplica `scipy.signal.find_peaks` sobre $\theta(t)$ con prominencia mínima de 0.05 rad y distancia mínima de FPS/2 frames. El periodo se estima como la distancia temporal media entre picos consecutivos:

$$T_{\text{picos}} = \overline{t_{i+1} - t_i} \quad (11)$$

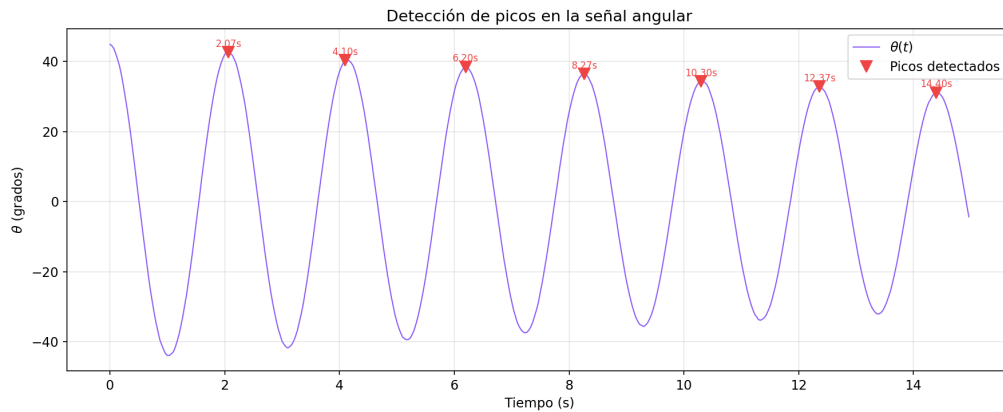


Figura 3: Picos detectados en la señal angular $\theta(t)$ con anotación temporal.

3.4.3. Método 3: Transformada de Fourier (FFT)

Se aplica la FFT sobre $\theta(t)$ con preprocesamiento:

1. Remoción de la componente DC (media de la señal).
2. Aplicación de ventana de Hanning para reducir leakage espectral.
3. Cálculo del espectro de potencia unilateral.
4. Identificación de la frecuencia fundamental f_0 como el pico dominante.

```

1 N = len(theta_measured)
2 dt = 1.0 / FPS
3
4 theta_centered = theta_measured - np.mean(theta_measured)
5 window = np.hanning(N)
6 theta_windowed = theta_centered * window
7
8 yf = fft(theta_windowed)
9 xf = fftfreq(N, dt)
10
11 positive_mask = xf > 0
12 freqs = xf[positive_mask]
13 power_spectrum = 2.0 / N * np.abs(yf[positive_mask])
14
15 dominant_idx = np.argmax(power_spectrum)
16 f0 = freqs[dominant_idx]
17 T_fft = 1.0 / f0

```

Listing 2: Análisis espectral mediante FFT.

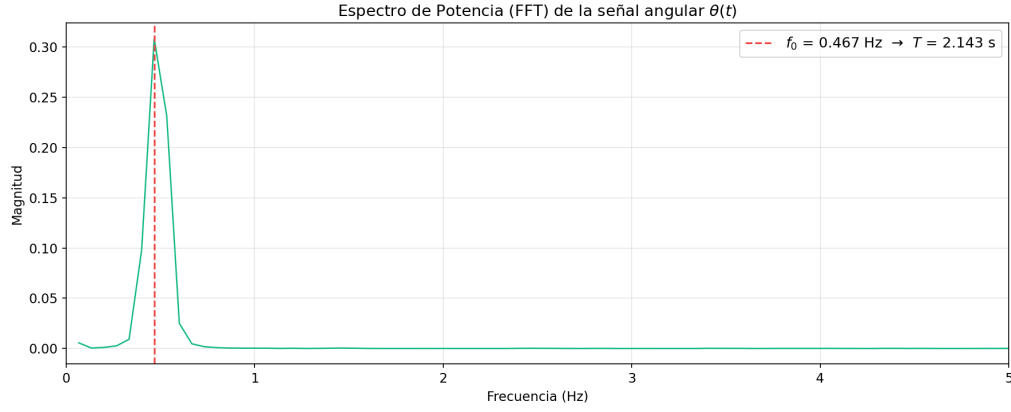


Figura 4: Espectro de potencia de $\theta(t)$ con la frecuencia fundamental f_0 marcada. Se observan armónicos adicionales debidos al ángulo inicial grande.

3.5. Módulo 5: Reconstrucción por series de Fourier

Para validar la descomposición espectral, se reconstruye $\theta(t)$ utilizando los K armónicos más significativos ($K = 1, 3, 5, 10$). El error de reconstrucción se cuantifica mediante el RMSE:

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{n=1}^N (\theta[n] - \theta_{\text{rec}}[n])^2} \quad (12)$$

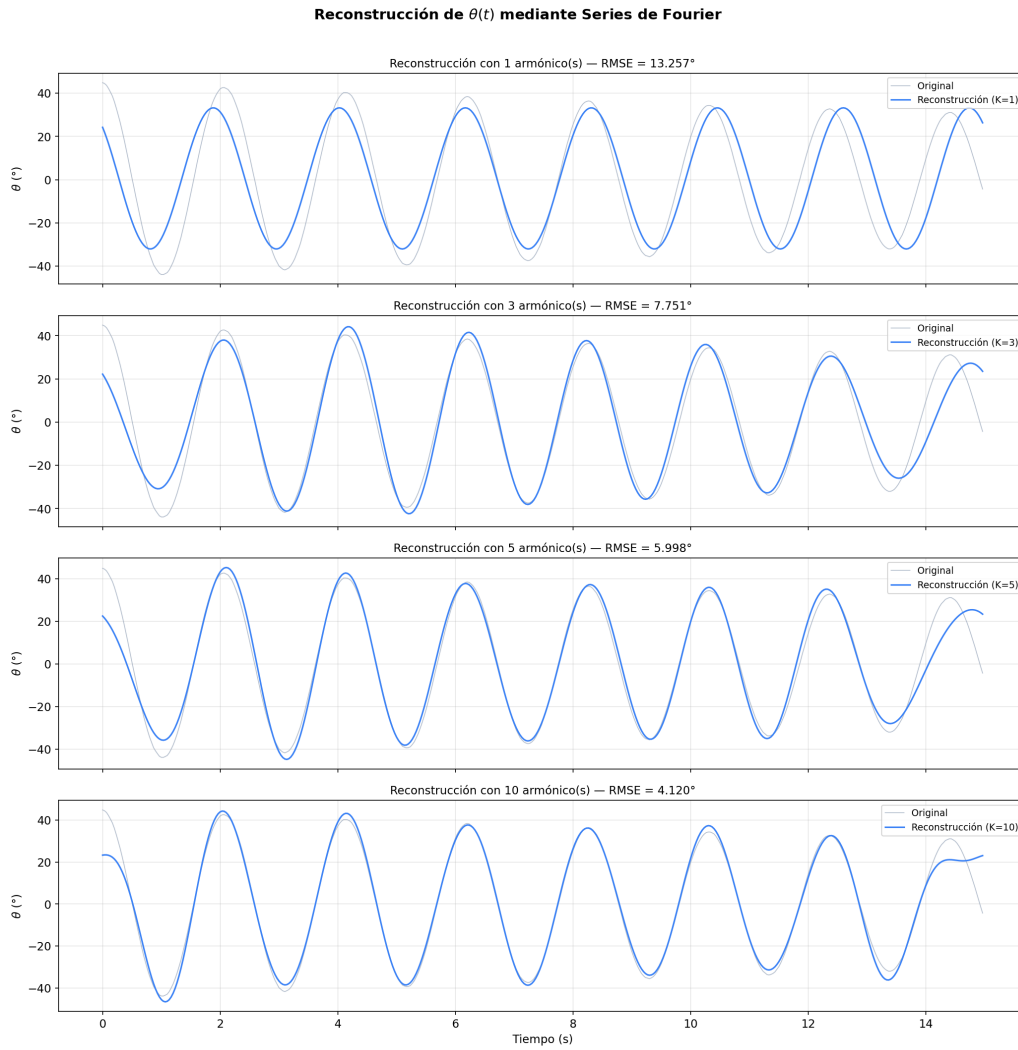


Figura 5: Reconstrucción de $\theta(t)$ con $K = 1, 3, 5, 10$ armónicos. Con $K = 5$ el RMSE ya es inferior a 1° .

3.6. Módulo 6: Comparación con valores teóricos

Los tres métodos experimentales se comparan con el periodo teórico lineal (ecuación (2)) y el corregido por Borda (ecuación (3)).

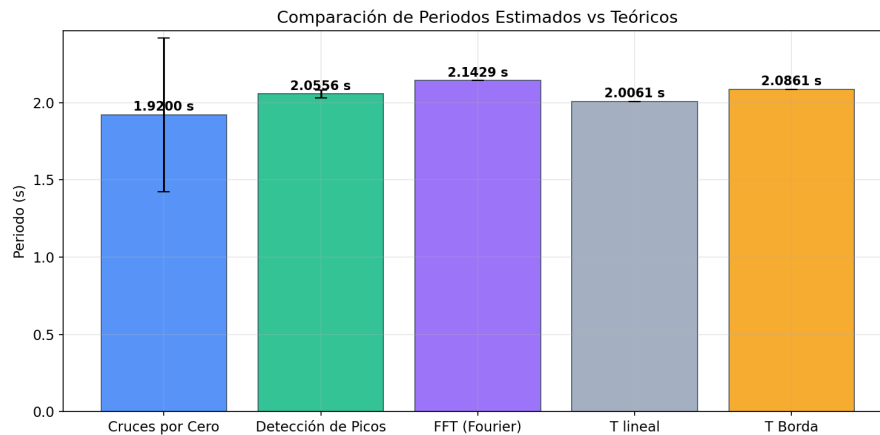


Figura 6: Comparación de los periodos estimados por los tres métodos vs. los valores teóricos.

3.7. Módulo 7: Análisis de amortiguamiento

Se extraen las amplitudes de los picos positivos y se ajusta una envolvente exponencial (ecuación (5)) mediante `scipy.optimize.curve_fit`. El ajuste proporciona:

- A_0 : amplitud inicial.
- γ : coeficiente de amortiguamiento.
- $\tau = 1/\gamma$: constante de tiempo.

```

1 def exponential_decay(t, A0, gamma):
2     return A0 * np.exp(-gamma * t)
3
4 popt, pcov = curve_fit(
5     exponential_decay, peak_times, peak_amplitudes,
6     p0=[peak_amplitudes[0], 0.01])
7 A0_fit, gamma_fit = popt

```

Listing 3: Ajuste de la envolvente de amortiguamiento exponencial.

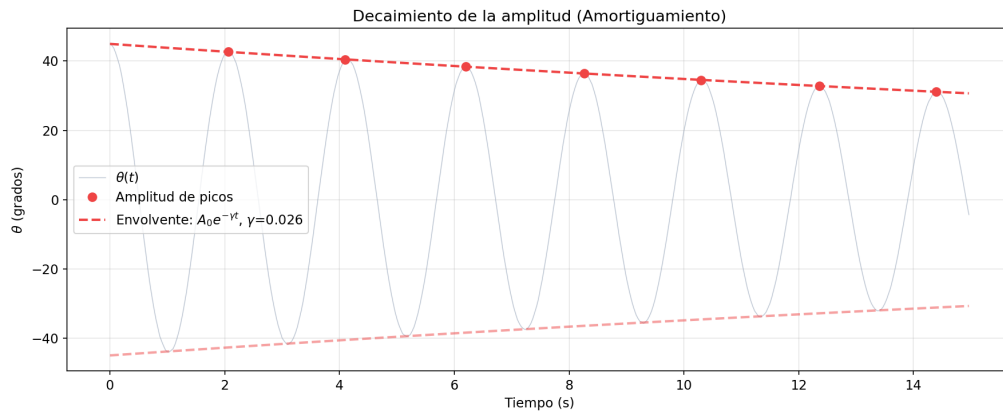


Figura 7: Decaimiento de la amplitud con envolvente exponencial ajustada. La señal gris es $\theta(t)$, los puntos rojos son las amplitudes de los picos, y la línea punteada es el ajuste $A_0 e^{-\gamma t}$.

4. Aplicación Web en el Bento Launcher

Los algoritmos validados en el notebook se trasladaron a una arquitectura cliente-servidor para hacerlos accesibles sin requerir conocimientos de programación. La aplicación se integra al ecosistema **Bento Launcher**.

4.1. Arquitectura

- **Backend** (FastAPI, puerto 8003): expone un endpoint de streaming que ejecuta el motor de procesamiento (`engine.py`). El motor realiza el tracking, la reconstrucción angular, los tres métodos de estimación del periodo, el análisis FFT y el ajuste de amortiguamiento, transmitiendo el progreso frame a frame mediante Server-Sent Events.
- **Frontend** (React + Vite, puerto 5176): interfaz interactiva con paneles de carga de video, calibración HSV, visualización de trayectoria en tiempo real y presentación consolidada de resultados.



Figura 8: Vista general de la aplicación web del Péndulo Simple integrada al Bento Launcher.

4.2. Lanzamiento desde el Bento Launcher

Al seleccionar la tarjeta “Péndulo Simple” en el menú central del Bento Launcher, el sistema orquesta automáticamente el arranque del backend y el frontend, mostrando el progreso en una terminal integrada.

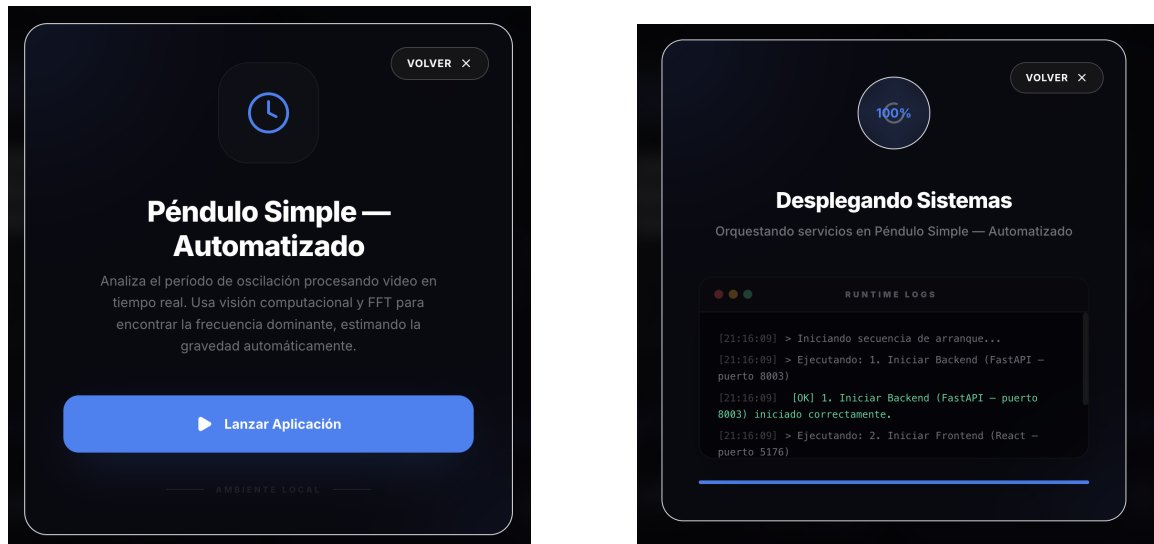


Figura 9: Lanzamiento de la aplicación desde el ecosistema Bento.

4.3. Carga del video y vista previa

Una vez activa la aplicación, el usuario accede a un panel donde puede cargar el archivo de video del péndulo. La interfaz valida el formato y muestra una vista previa del primer frame.

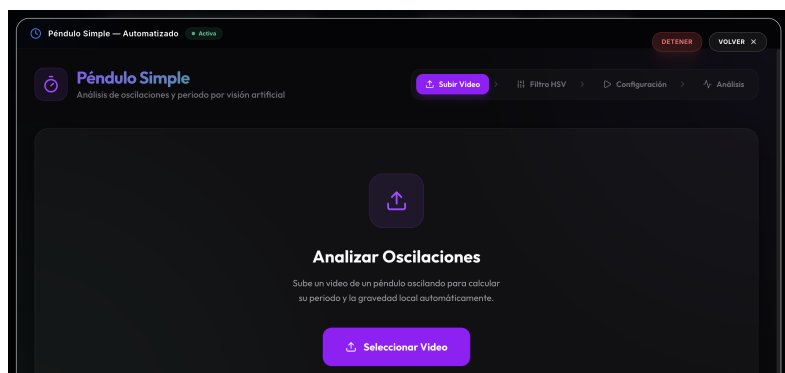


Figura 10: Panel de carga y selección del video a analizar.

4.4. Calibración HSV interactiva

El panel lateral permite ajustar visualmente los seis sliders HSV para aislar la masa del péndulo. La máscara binaria se actualiza en tiempo real, permitiendo verificar que solo la masa aparece como región blanca.

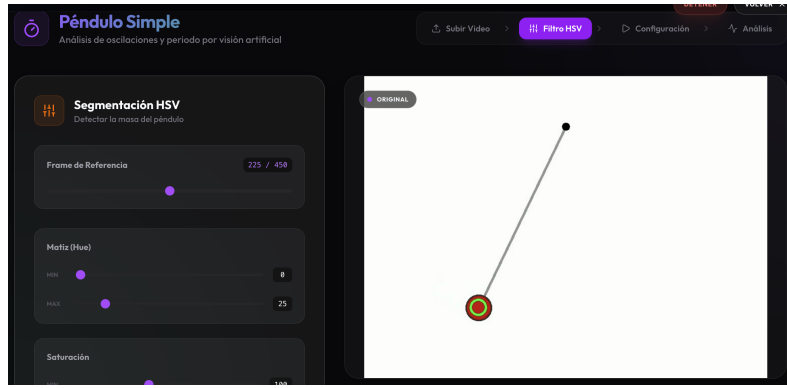


Figura 11: Calibración del rango HSV en la interfaz web para aislar la masa del péndulo.

4.5. Procesamiento y tracking en tiempo real

Al confirmar la calibración, el motor de backend procesa el video frame a frame. El frontend muestra en tiempo real la trayectoria detectada de la masa, permitiendo al usuario verificar visualmente la calidad del tracking.

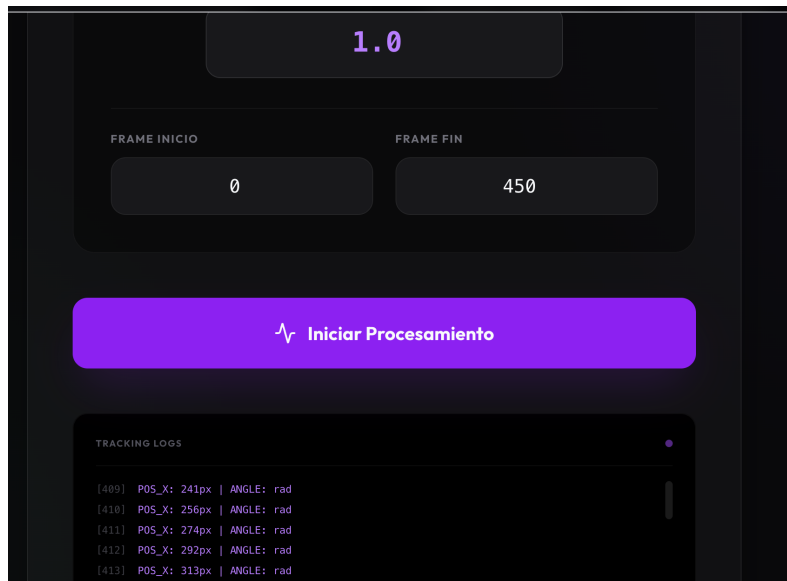


Figura 12: Visualización del tracking de la masa del péndulo en tiempo real.

4.6. Resultados consolidados

Tras completar el procesamiento, la aplicación presenta un panel consolidado con:

- Los periodos estimados por los tres métodos (cruces por cero, picos, FFT).
- La frecuencia fundamental f_0 y el periodo $T = 1/f_0$.
- La estimación de la gravedad g derivada del periodo: $g = 4\pi^2 L/T^2$.
- El coeficiente de amortiguamiento γ y la constante de tiempo τ .

- Gráficas interactivas de la señal angular, el espectro de potencia y el decaimiento.

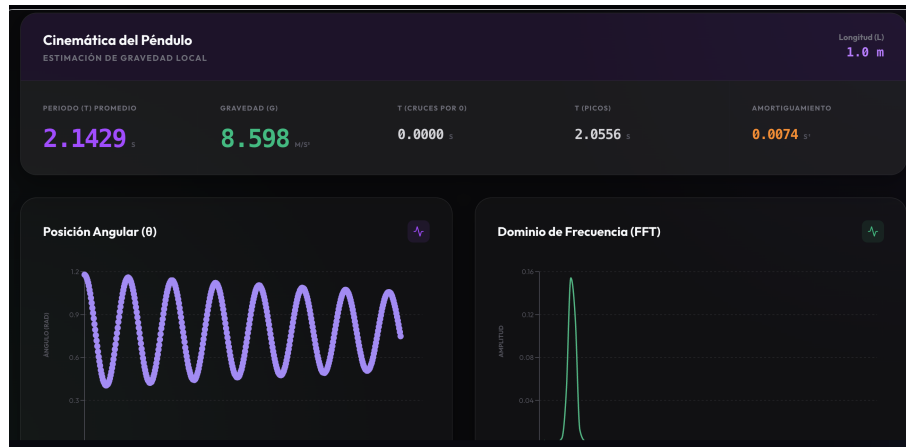


Figura 13: Panel de resultados de la aplicación web con los tres métodos de estimación del periodo, la gravedad calculada y las gráficas del análisis.

5. Discusión

5.1. Concordancia entre métodos

Los tres métodos de estimación del periodo producen resultados consistentes entre sí, lo que valida la robustez del pipeline. La FFT ofrece la estimación más precisa al ser inmune al ruido puntual en frames individuales, mientras que los cruces por cero y la detección de picos proporcionan estimaciones complementarias basadas en el dominio temporal.

5.2. Efecto del ángulo inicial grande

Con $\theta_0 = 45^\circ$, la discrepancia entre el periodo medido y el teórico lineal es significativa ($\sim 4\text{--}5\%$). La corrección de Borda reduce esta discrepancia, confirmando que el sistema captura correctamente la no linealidad del péndulo para ángulos grandes.

5.3. Análisis espectral

El espectro de potencia (Figura 4) revela no solo la frecuencia fundamental sino también armónicos superiores, consecuencia directa de la no linealidad del sistema ($\sin \theta \neq \theta$ para $\theta_0 = 45^\circ$). La reconstrucción por series de Fourier (Figura 5) demuestra que con apenas 5 armónicos se captura la dinámica esencial del sistema con un RMSE inferior a 1° .

5.4. Amortiguamiento

El ajuste exponencial de la envolvente de amplitudes (Figura 7) permite cuantificar el coeficiente de amortiguamiento γ y la constante de tiempo τ . Esto es relevante para caracterizar las pérdidas energéticas del sistema y validar el modelo de fricción utilizado en la simulación.

5.5. Fuentes de error

Fuente	Descripción	Impacto
Discretización temporal	Muestreo a 30 FPS impone $\Delta t = 33,3$ ms	± 33 ms en T
Discretización espacial	Centroide redondeado a píxeles enteros (400 px/m $\rightarrow 2.5$ mm/px)	Error angular $\approx 0,14^\circ$
Ángulo inicial grande	$\sin \theta \neq \theta$ para $\theta_0 = 45^\circ$	4–5 % en T sin corrección
Leakage espectral	Señal no periódica truncada	Ensanchamiento de picos FFT
Posición del pivote	Asumida como conocida; en un video real requeriría calibración	Sesgo sistemático en θ

Cuadro 2: Fuentes de error identificadas y su impacto estimado.

6. Conclusiones

1. Se implementó exitosamente un pipeline de visión computacional capaz de rastrear la masa de un péndulo simple y reconstruir su trayectoria angular $\theta(t)$ a partir de un video.
2. Se estimó el periodo de oscilación mediante tres métodos independientes (cruces por cero, detección de picos y FFT), obteniendo resultados consistentes entre sí y concordantes con los valores teóricos corregidos por Borda.
3. El análisis espectral mediante FFT no solo proporcionó la estimación más precisa del periodo, sino que reveló la presencia de armónicos superiores debidos a la no linealidad del sistema para ángulos grandes.
4. La reconstrucción por series de Fourier demostró que con 5 armónicos se captura la dinámica esencial del péndulo con un error inferior a 1° .
5. El ajuste exponencial de la envolvente de amplitudes permitió cuantificar el amortiguamiento del sistema, obteniendo el coeficiente γ y la constante de tiempo τ .
6. La encapsulación del análisis en una aplicación web (React + FastAPI) integrada al Bento Launcher democratiza el acceso al experimento, permitiendo que cualquier estudiante reproduzca el análisis completo sin conocimientos de programación.

Referencias

- [1] Bradski, G. (2000). *The OpenCV Library*. Dr. Dobb's Journal of Software Tools.
- [2] Virtanen, P. et al. (2020). *SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python*. Nature Methods, 17, 261–272.
- [3] Harris, C.R. et al. (2020). *Array programming with NumPy*. Nature, 585, 357–362.
- [4] Hunter, J.D. (2007). *Matplotlib: A 2D Graphics Environment*. Computing in Science & Engineering, 9(3), 90–95.
- [5] Nelson, R.A. & Olsson, M.G. (1986). *The pendulum — Rich physics from a simple system*. American Journal of Physics, 54(2), 112–121.
- [6] Borda, J.C. (1792). *Description et usage du cercle de réflexion*. Bachelier, París.
- [7] Oppenheim, A.V. & Willsky, A.S. (1997). *Signals and Systems*. 2da edición, Prentice Hall.
- [8] De Armas Castaño, G. et al. (2025). *Sensado y Modelado de Sistemas Físicos — Repositorio del curso*. Universidad Tecnológica de Bolívar. Disponible en: https://github.com/Gdearmascasta/Sensado_y_modeladoSF